

ARCADE

Epitech Project – 2026

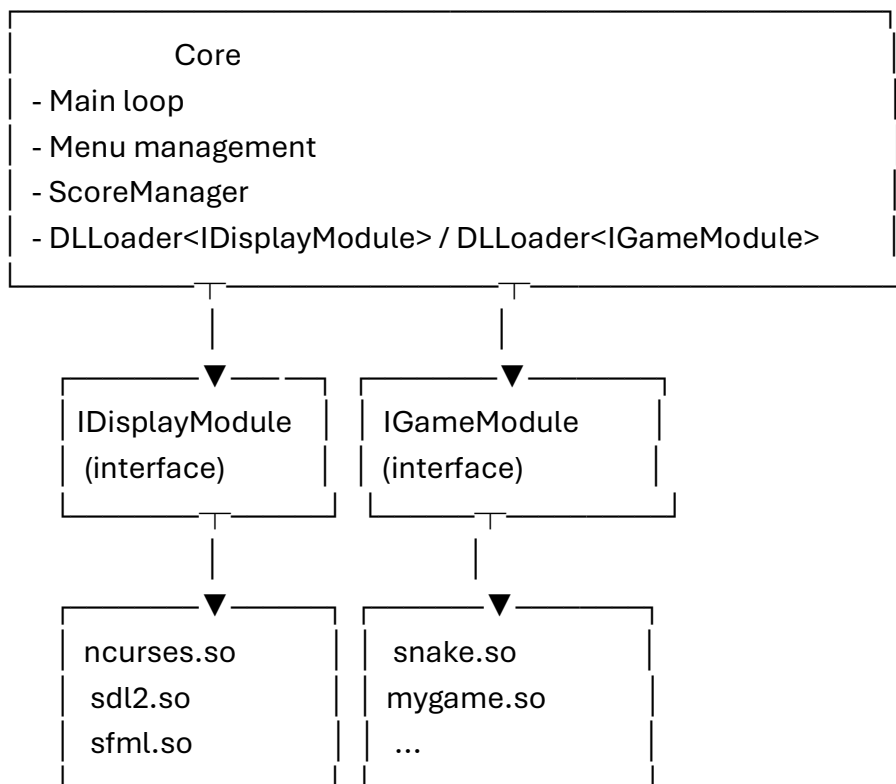
Sommaire

- Global Architecture_____2
- Adding a Graphical Library_____6
- Adding a game_____11
- RenderData — Game ↔ Display Communication_____17

Global Architecture

Overview

The Arcade project is built on three distinct layers that communicate through abstract C++ interfaces and dynamic library loading (.so).



The Core

Arcade::Core is the orchestrator. It:

1. **Scans** `./lib/` at startup to list available .so files (graphical libs are identified by the keywords `ncurses`, `sdl2`, `SFML`; everything else is treated as a game).
2. **Dynamically loads** the initial graphical library passed as a command-line argument.
3. **Drives the main loop:** poll events → update game → render.
4. **Allows hot-switching** of graphical library or game at runtime (`NEXT_LIB`, `PREV_LIB`, `NEXT_GAME`, `PREV_GAME` keys).

```
// Core.cpp — simplified main loop
while (_running) {
    Key key = _graphicalModule->pollEvent();
    handleSystemEvents(key);
    _gameModule->handleInput(key);
    _gameModule->update();
    _graphicalModule->clear();
    _graphicalModule->render(_gameModule->getRenderData());
    _graphicalModule->display();
}
```

DLLoader

DLLoader<T> is a wrapper around dlopen / dlsym / dlclose.

It loads a .so library and extracts a C symbol (factory function) to instantiate the module.

```
// Loading a graphical library
_graphicalLoader = std::make_unique<DLLoader<IDisplayModule>>(path);
_graphicalModule = _graphicalLoader->getInstance("createDisplayModule");

// Loading a game
_gameLoader = std::make_unique<DLLoader<IGameModule>>(path);
_gameModule = _gameLoader->getInstance("createGameModule");
```

Interfaces

IDisplayModule

Represents a graphical library. Required methods:

Method	Role
init()	Window / context initialization
close()	Clean shutdown
clear()	Clear the screen
render(RenderData)	Draw the current frame
display()	Present frame (buffer swap / refresh)

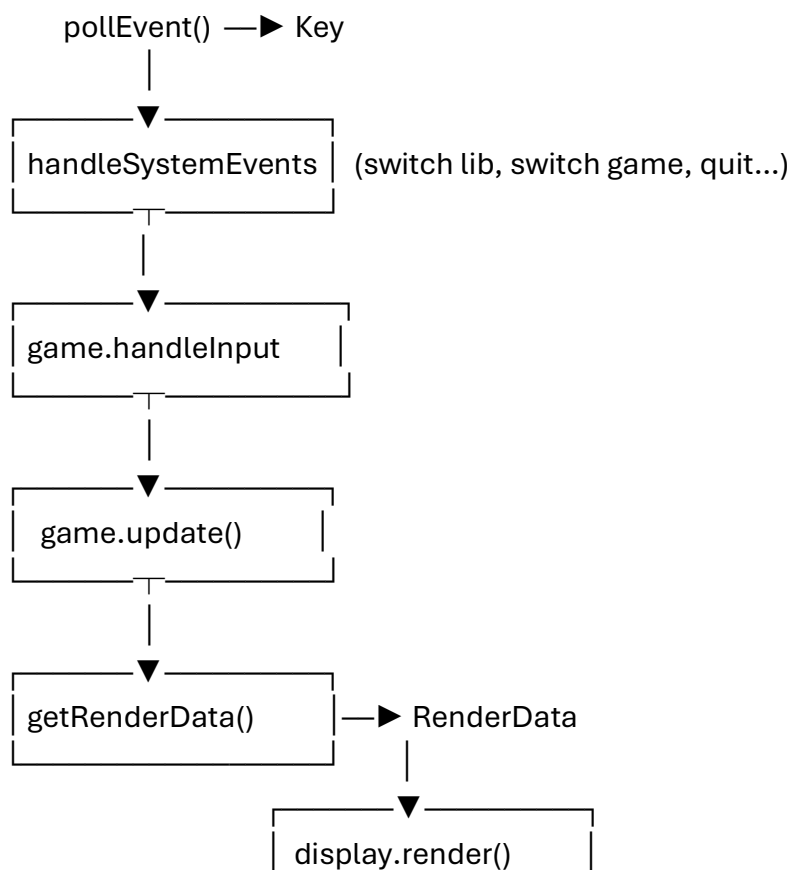
pollEvent()	Read input → return Arcade::Key
getName()	Library name

IGameModule

Represents a game. Required methods:

Method	Role
init()	Initialize game state
reset()	Reset to initial state
close()	Clean up resources
handleInput(Key)	Receive a key press
update()	Advance one game tick
getRenderData()	Return the current RenderData
isGameOver()	true when the game is over
getScore()	Current score
getName()	Game name

Frame Lifecycle





Reserved System Keys

Key	Action
ESCAPE	Return to menu
QUIT	Exit the program
NEXT_LIB / PREV_LIB	Switch graphical library
NEXT_GAME / PREV_GAME	Switch game
RESTART	Restart the current game

Adding a Graphical Library

Prerequisites

- Understand the IDisplayModule interface (see include/IDisplayModule.hpp)
- Have a compatible build system (Makefile, CMake...)
- The library must compile as a shared .so

Step 1 — Create source files

```
src/  
└─ display/  
    └─ MyLib/  
        ├── MyLibDisplay.hpp  
        └─ MyLibDisplay.cpp
```

Step 2 — Implement IDisplayModule

MyLibDisplay.hpp

```
#pragma once  
#include "IDisplayModule.hpp"  
#include <string>  
  
class MyLibDisplay : public Arcade::IDisplayModule {  
public:  
    MyLibDisplay();  
    ~MyLibDisplay() override = default;  
  
    void    init()    override;  
    void    close()   override;  
    void    clear()   override;
```

```

void    render(const Arcade::RenderData &data) override;
void    display() override;
Arcade::Key pollEvent() override;
const std::string &getName() const override;

private:
    std::string _name;
    // your private members here (window, renderer, etc.)
};

```

MyLibDisplay.cpp — minimal skeleton

```

#include "MyLibDisplay.hpp"

MyLibDisplay::MyLibDisplay() : _name("mylib") {}

void MyLibDisplay::init() {
    // Open the window, initialize the graphical context
}

void MyLibDisplay::close() {
    // Clean window shutdown
}

void MyLibDisplay::clear() {
    // Clear the current buffer
}

void MyLibDisplay::render(const Arcade::RenderData &data) {
    // Iterate over data.grid and data.texts, draw each element
    for (int y = 0; y < data.gridHeight; ++y) {
        for (int x = 0; x < data.gridWidth; ++x) {
            const Arcade::Cell &cell = data.grid[y][x];
            // Draw cell.character at position (x, y)
            // using cell.fgColor as foreground color
        }
    }
    for (const auto &text : data.texts) {
        // Draw text.content at (text.x, text.y)
        // using text.color as color
    }
}

```

```

    }
}

void MyLibDisplay::display() {
    // Buffer swap / refresh / present frame
}

Arcade::Key MyLibDisplay::pollEvent() {
    // Read keyboard event and translate it to Arcade::Key
    // Return Arcade::Key::NONE if no event is pending
    return Arcade::Key::NONE;
}

const std::string &MyLibDisplay::getName() const {
    return _name;
}

```

Step 3 — Export the C symbol (mandatory)

The Core looks for the symbol `createDisplayModule` via `dlsym`.

You **must** export it with `extern "C"` to prevent C++ name mangling:

```

// At the end of MyLibDisplay.cpp (or in a dedicated file)
extern "C" Arcade::IDisplayModule *createDisplayModule() {
    return new MyLibDisplay();
}

```

Important: the symbol name must be exactly `createDisplayModule`.

Step 4 — Compile as a .so

With g++ directly

```

g++ -shared -fPIC -o lib/arcade_mylib.so \
    src/display/MyLib/MyLibDisplay.cpp \

```

```
-I include/\  
-lmylib_dependency # add your graphical library flags here
```

With a Makefile (example)

```
LIB_NAME = lib/arcade_mylib.so  
SRC      = src/display/MyLib/MyLibDisplay.cpp  
CXXFLAGS = -std=c++20 -Wall -Wextra -fPIC -I include/  
LDFLAGS  = -shared -lmylib_dependency
```

```
$(LIB_NAME): $(SRC)  
    $(CXX) $(CXXFLAGS) $(LDFLAGS) -o $@ $^
```

Step 5 — Place in ./lib/

The filename **must** contain one of the following keywords to be recognized as a graphical library (rather than a game):

- ncurses
- sdl2
- sfml

```
cp lib/arcade_mylib.so ./lib/arcade_sdl2.so # example for SDL2
```

If you add a new technology, update scanLibraries() in Core.cpp to include your keyword.

Summary — Checklist

- [] Class inheriting from IDisplayModule
- [] All interface methods implemented
- [] C symbol createDisplayModule exported with extern "C"
- [] Compiled with -shared -fPIC
- [] The .so file is placed in ./lib/
- [] The filename contains ncurses, sdl2, or sfml

Adding a Game

Prerequisites

- Understand the IGameModule interface (see include/IGameModule.hpp)
- Understand the RenderData format (see [04_render_data.md](#))

Step 1 — Create source files

```
src/  
└─ games/  
    └─ MyGame/  
        ├── MyGame.hpp  
        └─ MyGame.cpp
```

Step 2 — Implement IGameModule

MyGame.hpp

```
#pragma once  
#include "IGameModule.hpp"  
#include <string>  
  
class MyGame : public Arcade::IGameModule {  
  
public:  
    MyGame();  
    ~MyGame() override = default;  
  
    void    init()            override;  
    void    reset()           override;  
    void    close()           override;  
    void    handleInput(Arcade::Key key) override;  
    void    update()          override;
```

```

const Arcade::RenderData &getRenderData()    override;
bool    isGameOver() const    override;
int     getScore() const      override;
const std::string &getName() const    override;

private:
    std::string    _gameName;
    Arcade::RenderData _renderData;
    bool    _gameOver;
    int     _score;
    int     _gridWidth;
    int     _gridHeight;

    // Your game state here
    void buildRenderData();
};

```

MyGame.cpp — minimal skeleton

```

#include "MyGame.hpp"

MyGame::MyGame()
    : _gameName("MyGame"), _gameOver(false), _score(0),
      _gridWidth(20), _gridHeight(20)
{}

void MyGame::init() {
    _gameOver = false;
    _score    = 0;
    // Initialize your game state
    buildRenderData();
}

void MyGame::reset() {
    init();
}

void MyGame::close() {
    // Free any resources
}

```

```

void MyGame::handleInput(Arcade::Key key) {
    if (key == Arcade::Key::UP) { /* move up */ }
    if (key == Arcade::Key::DOWN) { /* move down */ }
    if (key == Arcade::Key::LEFT) { /* move left */ }
    if (key == Arcade::Key::RIGHT) { /* move right */ }
}

void MyGame::update() {
    if (_gameOver) return;
    // Game logic: move entities, detect collisions...
    // On losing condition:
    // _gameOver = true;
    buildRenderData();
}

const Arcade::RenderData &MyGame::getRenderData() {
    return _renderData;
}

bool MyGame::isGameOver() const {
    return _gameOver;
}

int MyGame::getScore() const {
    return _score;
}

const std::string &MyGame::getName() const {
    return _gameName;
}

// — Build RenderData

```

```

void MyGame::buildRenderData() {
    _renderData.gridWidth = _gridWidth;
    _renderData.gridHeight = _gridHeight;

    // Fill the grid
    _renderData.grid.clear();
}

```

```

for (int y = 0; y < _gridHeight; ++y) {
    std::vector<Arcade::Cell> row;
    for (int x = 0; x < _gridWidth; ++x) {
        Arcade::Cell cell;
        cell.character = ' ';
        cell.fgColor = Arcade::Color::WHITE;
        cell.bgColor = Arcade::Color::BLACK;
        row.push_back(cell);
    }
    _renderData.grid.push_back(row);
}

// Display score as text
_renderData.texts.clear();
Arcade::Text scoreText;
scoreText.content = "Score: " + std::to_string(_score);
scoreText.x = 0;
scoreText.y = _gridHeight + 1;
scoreText.color = Arcade::Color::WHITE;
_renderData.texts.push_back(scoreText);
}

```

Step 3 — Export the C symbol (mandatory)

```

// At the end of MyGame.cpp
extern "C" Arcade::IGameModule *createGameModule() {
    return new MyGame();
}

```

Important: the symbol name must be exactly createGameModule.

Step 4 — Compile as a .so

With g++ directly

```
g++ -shared -fPIC -o lib/arcade_mygame.so \  
    src/games/MyGame/MyGame.cpp \  
    -I include/
```

With a Makefile (example)

```
LIB_NAME = lib/arcade_mygame.so  
SRC      = src/games/MyGame/MyGame.cpp  
CXXFLAGS = -std=c++20 -Wall -Wextra -fPIC -I include/  
LDFLAGS  = -shared
```

```
$(LIB_NAME): $(SRC)  
    $(CXX) $(CXXFLAGS) $(LDFLAGS) -o $@ $^
```

Step 5 — Place in ./lib/

The filename must **not** contain ncurses, sdl2, or sfml (those keywords identify graphical libraries).

```
cp lib/arcade_mygame.so ./lib/
```

The Core will detect it automatically on the next launch.

Game Over Handling

When `isGameOver()` returns true, the Core:

1. Records the score via `ScoreManager` (using `getName()` and `getScore()`)
2. Returns to the menu

Your game **must not** handle the return to menu itself.

Summary — Checklist

- [] Class inheriting from IGameModule
- [] All interface methods implemented
- [] getRenderData() returns a valid RenderData every frame
- [] isGameOver() returns true when the game is over
- [] C symbol createGameModule exported with extern "C"
- [] Compiled with -shared -fPIC
- [] The .so file is in ./lib/ without graphical keywords in the name

RenderData — Game ↔ Display Communication

RenderData is the **only structure exchanged** between a game (IGameModule) and a graphical library (IDisplayModule). It is defined in include/RenderData.hpp (or equivalent).

Full Structure

```
namespace Arcade {

    // Available colors
    enum class Color {
        BLACK,
        WHITE,
        RED,
        GREEN,
        YELLOW,
        BLUE,
        MAGENTA,
        CYAN
    };

    // A single cell in the game grid
    struct Cell {
        char character; // Character to display ('o', '#', ' ', ...)
        Color fgColor;  // Foreground (character) color
        Color bgColor;  // Background color
    };

    // A text element displayed outside the grid (HUD, score, game over...)
    struct Text {
        std::string content; // Text content
        int    x;           // Column (in character units)
        int    y;           // Row
        Color   color;      // Text color
    };

    // Main structure exchanged between game and display
    struct RenderData {
        int          gridWidth; // Grid width
        int          gridHeight; // Grid height
        std::vector<std::vector<Cell>> grid; // Grid [y][x]
        std::vector<Text> texts; // Text overlays
    };
}
```

```
}
```

The Grid (grid)

The grid is a 2D vector of Cell, indexed as grid[y][x].

- The **Y** axis goes from top (0) downward.
- The **X** axis goes from left (0) rightward.
- Each cell holds a character and its colors.

Example — Snake

Character	Meaning	Color
#	Wall	BLUE
@	Snake head	YELLOW
o	Snake body	GREEN
*	Food	RED
	Empty cell	WHITE

```
// Place a wall along the top of the grid
for (int x = 0; x < gridWidth; ++x) {
    _renderData.grid[0][x].character = '#';
    _renderData.grid[0][x].fgColor = Arcade::Color::BLUE;
}
```

Texts (texts)

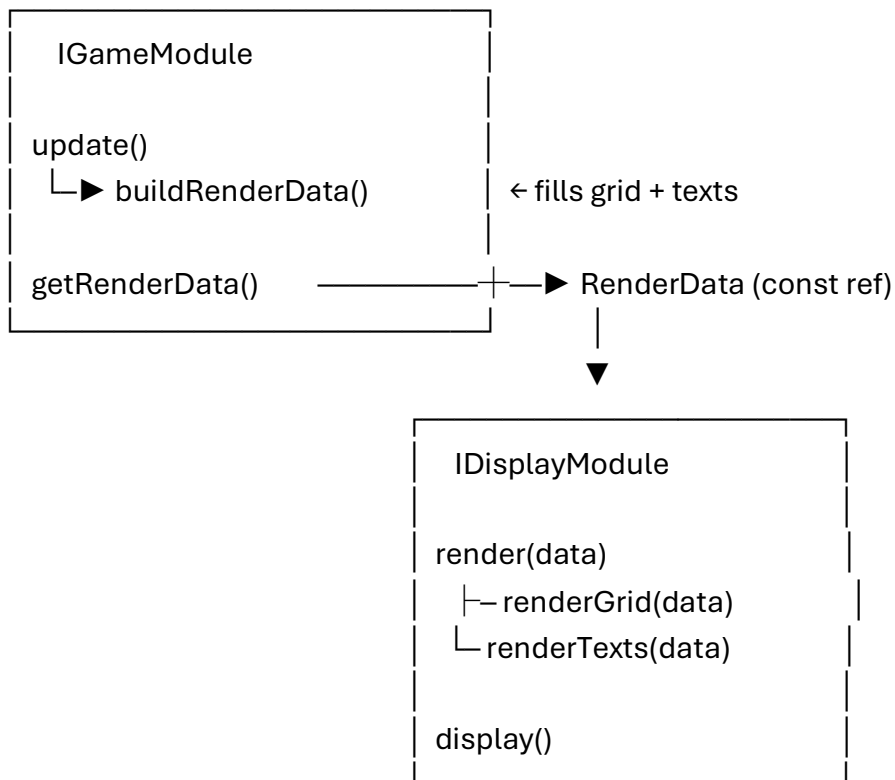
Texts allow you to display the HUD (score, game name, game over...) **below** or **overlaid on** the grid.

Position (x, y) is in **character units**. By convention, place texts below the grid:

```
Arcade::Text scoreText;
scoreText.content = "Score: " + std::to_string(_score);
scoreText.x = 0;
scoreText.y = _gridHeight + 1; // just below the grid
scoreText.color = Arcade::Color::WHITE;
```

```
_renderData.texts.push_back(scoreText);
```

RenderData Lifecycle



1. The game updates its internal state in `update()`.
2. It encodes that state into `_renderData` via `buildRenderData()`.
3. The Core calls `getRenderData()` → the graphical library receives a const ref.
4. The graphical library iterates over grid and texts to draw.
5. `display()` presents the frame on screen.

Implementation Tips

- **Rebuild `_renderData` on every `update()`:** it is simple and avoids hidden state.
- **Do not store pointers** to `RenderData` inside the graphical library: it is a temporary reference valid only until the next `update()`.
- **Always clear the entire grid** with spaces before drawing entities, otherwise artifacts from the previous frame may persist.

```
// Proper grid initialization
void MyGame::buildRenderData() {
    _renderData.gridWidth = _gridWidth;
    _renderData.gridHeight = _gridHeight;
    _renderData.grid.clear();
    _renderData.texts.clear();

    for (int y = 0; y < _gridHeight; ++y) {
        std::vector<Arcade::Cell> row(_gridWidth);
        for (auto &cell : row) {
            cell.character = ' ';
            cell.fgColor = Arcade::Color::WHITE;
            cell.bgColor = Arcade::Color::BLACK;
        }
        _renderData.grid.push_back(row);
    }

    // ... draw entities on top ...
}
```